

Contents

1	Introduction	2
1.1	Objectives	2
2	Greedy Approaches	4
2.1	Analysis	4
3	Integer Programming	5
3.1	Mathematical Model	5
3.2	OR-Tools	6
4	Constraint Programming	7
4.1	Constraint Optimization Problem	7
4.2	AC-3	8
4.3	OR-Tools	10
5	Extensions	11
5.1	Coloring	11
5.2	Extended Visibility	13
6	Conclusion	14

Chapter 1

Introduction

In this report, a geometric coverage problem is presented and solved using various combinatorial optimization techniques. The problem consists of a set of rectangles and their respective vertices. The primary objective is to deploy a set of guards strictly on these vertices such that every rectangle is covered. A rectangle is considered covered if at least one of its incident vertices possesses a guard. Consequently, the initial problem is to determine a vertex assignment that minimizes the total number of guards required.

Subsequently, an extension to this baseline problem is explored. In practical contexts, guards may represent resources that require differentiation, preventing interference or scheduling conflicts. Therefore, a secondary objective is introduced, which consists of assigning a color to each placed guard such that no rectangle is covered by more than one guard of the same color, while minimizing the total number of colours utilized. This formulation effectively transforms the problem into a multi-objective optimization problem.

The following chapters establish mathematical formulations using Greedy Approaches, Integer Programming, and Constraint Programming. Then, the implementation of these approaches using Google OR-Tools and an independent search method maintaining arc-consistency with AC-3 is described. Finally, the multi-objective extension is formulated and solved using a sequential optimization method.

1.1 Objectives

The primary objectives of this report are defined as follows:

1. To evaluate Greedy Approaches as a preliminary method for the guard placement problem.
2. To establish rigorous mathematical formulations for the problem utilizing both Integer Programming and Constraint Programming models.
3. To implement the proposed mathematical models utilizing the Google OR-Tools suite.
4. To develop and evaluate a custom search algorithm employing AC-3 for information filtering and backtracking to solve the Constraint Optimization Problem.
5. To formulate and solve a multi-objective extension of the problem, focusing on the minimization of the colours assigned to the minimum number of guards found in the initial models.

Chapter 2

Greedy Approaches

2.1 Analysis

Chapter 3

Integer Programming

In this chapter, an Integer programming model is presented. The initial step in this process is to establish the mathematical formulation of the problem, which includes the definition of the decision variables, constraints, and objective function. Then, the implementation of this approach using Google OR-Tools is described, followed by an interpretation of the results.

3.1 Mathematical Model

Let m be the number of rectangles to cover, n the number of vertices and p_{ij} represented as follows:

$$p_{ij} = \begin{cases} 1, & \text{if rectangle } j \text{ is incident to vertex } i \\ 0, & \text{otherwise} \end{cases}$$

Decision Variables: v_i indicates wheter vertex i has a guard or not.

The objective is to minimize the total number of guards:

$$\sum_{i=1}^n v_i$$

subject to the following constraints:

$$\begin{cases} \sum_{i=1}^n v_i \times p_{ij} \geq 1, & \forall j = 1, 2, \dots, m \\ v_i \in \{0, 1\}, & \forall i = 1, 2, \dots, n \end{cases}$$

The objective of this constraint is to ensure that each rectangle is covered by ensuring that at least one of its vertices possesses a guard.

3.2 OR-Tools

The implementation of the Integer Programming model utilizes the `MPSolver` interface from Google OR-Tools, specifically configured to use the SCIP (Solving Constraint Integer Programs) backend.

The implementation translates the mathematical formulation directly into C++ code:

- **Variables:** Using `solver→MakeIntVar(0.0, 1.0, varName)`, boolean decision variables x_i are instantiated for each candidate vertex in the problem space.
- **Constraints:** For each rectangle, a row constraint is created via `solver→MakeRowConstraint(1.0, inf)`. The variables corresponding to the vertices incident to that rectangle are assigned a coefficient of 1. This enforces the condition $\sum_{i=1}^n x_i \times p_{ij} \geq 1$.
- **Objective Function:** The objective function is defined by setting the coefficient of every vertex variable to 1 via `objective→SetCoefficient(x[i], 1)` and invoking `objective→SetMinimization()` to ensure the solver finds the minimum set of guards.

Additionally, to improve solver performance by reducing the search space, a theoretical lower bound for the number of guards needed is calculated. Specifically, a bound of $\lceil m/3 \rceil - 1$ (where m is the number of rectangles to be covered) is computed and added as a global row constraint via `solver→MakeRowConstraint(solutionLowerbound - 1, inf)`. This prevents the solver from evaluating unfeasible sub-optimal branches.

Chapter 4

Constraint Programming

In this chapter, a Constraint programming model is presented. The initial step in this process is to establish the Constraint Optimization Problem (COP). Then, we discuss how to solve the COP using two different methods: maintaining arc-consistency with AC-3 and using OR-Tools.

4.1 Constraint Optimization Problem

A Constraint Optimization Problem is a tuple (X, D, C) where:

- $X = \{x_1, x_2, \dots, x_n\}$ is the set of variables;
- $D = \{d_1, d_2, \dots, d_n\}$ is the set of domains, where d_i is a finite set of possible values for x_i ;
- $C = \{c_1, c_2, \dots, c_m\}$ is the set of constraints;
- f is the optimization function.

The following COP can be used to model this problem:

- $X = \{v_1, v_2, \dots, v_n\}$: set of vertices;
- $D = \{d_1, d_2, \dots, d_n\} : d_1 = d_2 = \dots = d_n = \{0, 1\}$;
- $C: \sum_{i=1}^n v_i \times p_{ij} \geq 1 \in C, \quad \forall j = 1, 2, \dots, m$;
- $f = \sum_{i=1}^n v_i$.

The model under discussion is essentially analogous to the model for integer programming, but expressed in the form of a COP.

4.2 AC-3

In this section, the methodology for solving the COP mentioned in the previous section, by maintaining arc consistency, will be demonstrated using AC-3 as a mean of filtering and propagating information.

Two different approaches were considered with regard to the AC-3. Initially, it seemed a good idea to perform a dual graph translation to obtain the corresponding binary COP. However, this would result in exponential complexity for the translation and a combinatorial explosion in the number of constraints in the resulting binary formulation. Therefore, we opted for a generalised version of the AC-3, which, although still computationally expensive, does not require an initial translation step.

The following pseudocode describes the AC-3 algorithm:

```

procedure AC3( $X, D, C$ )
   $Q \leftarrow \{(i, c) \mid c \in C, x_i \in \text{scope}(c)\}$ 
  while  $Q \neq \emptyset$  do
     $(i, c) \leftarrow \text{Fetch}(Q)$ 
    if REVISE( $i, c$ ) then
       $Q \leftarrow Q \cup \{(j, c') \mid c' \in C, c' \neq c, j \neq i, \{x_i, x_j\} \subseteq \text{scope}(c')\}$ 
    end if
  end while
end procedure

function REVISE( $i, c$ )
   $change \leftarrow false$ 
  for each  $a \in d_i$  do
    if  $\forall a_1 \in d_1, \dots, a_{i-1} \in d_{i-1}, a_{i+1} \in d_{i+1}, \dots, a_k \in d_k, \neg c(x_1 \leftarrow$ 
 $a_1, \dots, x_i \leftarrow a, \dots, x_k \leftarrow a_k)$  then
      remove  $a$  from  $d_i$ 
       $change \leftarrow true$ 
    end if
  end for
  return  $change$ 
end function

```

Although AC-3 itself is employed for the filtration of information, a method of propagating the filtered results is still required. The search algorithm that has been selected to work in conjunction with AC-3 is known as backtracking.

The following pseudocode describes the backtracking algorithm:


```

function BT( $\tau, X, D, C$ )
  if  $X$  is empty then
    return true
  end if
  if  $\neg AC3(X, D, C)$  then
    return false
  end if
   $x_i \leftarrow \text{Select}(X)$ 
  for each  $a \in \text{Domain}(x_i)$  do
     $\tau(x_i) \leftarrow a$ 
    if consistent( $\tau, C, x_i$ ) then
      if BT( $\tau, X, D[d_i \rightarrow \{a\}], C$ ) then
        return true
      end if
    end if
    restore state or use persistent data structures
  end for
  return false
end function

```

```

function CONSISTENT( $\tau, C, x_i$ )
  for each  $c \in C$  s.t.  $\text{scope}(c) \subseteq \text{vars}(\tau)$  and  $x_i \in \text{scope}(c)$  do
    if  $\neg c(\tau)$  then
      return false
    end if
  end for
  return true
end function

```

As AC-3 and backtracking only determine the existence of a feasible solution within a given bound, an additional optimisation strategy is required to minimise the objective function. Therefore, we perform a binary search over the objective value. Let m be the upper bound for the number of selected vertices. A new constraint is temporarily added to the CPO.

If no solution is found, the lower bound variable is updated. Otherwise, a feasible solution has been found and the upper bound variable can be reduced. This process continues until the optimal value is identified. At the end, the optimal value is stored in the lower bound variable l .

```

procedure BINARYSEARCH( $X, D, C$ )
   $l \leftarrow 0$ 
   $r \leftarrow m$ 
  while  $l < r$  do
     $m \leftarrow l + (r - l)/2$ 

```

```

    add constraint  $f \leq m$ 
    if BT( $\tau, X, D, C$ ) then
         $r \leftarrow m$ 
    else
         $l \leftarrow m + 1$ 
    end if
end while
return  $l$ 
end procedure

```

4.3 OR-Tools

To implement the Constraint Optimization Problem, the OR-Tools CP-SAT solver is employed via the `CpModelBuilder` API. The CP-SAT solver utilizes satisfiability (SAT) methods under the hood to efficiently explore the domain.

In accordance with the base problem formulation, the model construction focuses solely on the optimal placement of guards:

- **Variables:** The decision variables v_i are initialized using `cp_model.NewIntVar(0, 1, name)`, strictly bounding the domain of each candidate vertex to $\{0, 1\}$.
- **Constraints:** The coverage constraints are constructed using linear expressions (`LinearExpr`). For each rectangle, the boolean variables of its incident vertices are summed. The constraint is then strictly enforced using `cp_model.AddGreaterOrEqual(expr, 1)`, ensuring every rectangle is covered by at least one guard.
- **Objective:** The objective is defined as minimizing the total number of placed guards. This is achieved by summing all v_i variables and applying `cp_model.Minimize(expr)`.

The model is compiled and solved using `Solve(cp_model.Build())`. The solver returns a `CpSolverResponse`, which yields the optimal geometric configuration of guards that minimizes the total count.

Chapter 5

Extensions

In this chapter, two extensions of the original problem are presented.

5.1 Coloring

Given the minimum number of guards required to cover all rectangles, the objective is to assign colours to the guards such that guards covering the same rectangle have different colours, while minimising the total number of colours used.

This problem belongs to the class of multi-objective optimisation problems, since the objective is to minimise both the number of guards used and the number of colours used. This finding suggests that one minimization is considerably more significant than the other. It is therefore proposed that a weighted-sum method be employed. However, following a review of the relevant online literature, it became apparent that proving the veracity of our model and defining a weight for each constraint would be a challenge. Consequently, a sequential optimisation method was employed, whereby multiple objectives were transformed into single-objective sub-problems. The first objective was the minimisation of the guards, and the second was the minimisation of the colours, with an additional restriction that the number of guards used must be equal to the minimum found in the previous model.

The first optimisation problem, concerning the minimisation of the number of guards, was presented in the previous chapters. Consequently, the present chapter is exclusively concerned with the second optimisation problem.

Let m be the number of rectangles to cover, n the number of vertices, s the number of guards to use and z be the maximum number of colors, that is, n . Let

$$p_{ij} = \begin{cases} 1, & \text{if rectangle } j \text{ is incident to vertex } i \\ 0, & \text{otherwise} \end{cases}$$

The decision variables are defined as follows:

- v_i indicates if vertex i contains a guard;
- u_{ij} indicates if vertex i has color j ;
- c_j indicates if color j is used.

The objective is to minimize the number of guard types used:

$$\min \sum_{i=1}^z c_i$$

subject to the following constraints:

$$\left\{ \begin{array}{l} \sum_{i=1}^n v_i \times p_{ij} \geq 1, \quad \forall j = 1, 2, \dots, m \\ \sum_{i=1}^n v_i = s \\ \sum_{j=1}^z u_{ij} = v_i, \quad \forall i = 1, 2, \dots, n \\ \sum_{i=1}^n u_{ij} \geq c_j, \quad \forall j = 1, 2, \dots, z \\ \sum_{i=1}^n p_{ir} \times u_{ik} \leq 1, \quad \forall r = 1, 2, \dots, m, \forall k = 1, 2, \dots, z \\ u_{ij} \leq c_j, \quad \forall i = 1, 2, \dots, n, \forall j = 1, 2, \dots, z \\ v_i \in \{0, 1\}, \quad \forall i = 1, 2, \dots, n \\ u_{ij} \in \{0, 1\}, \quad \forall i = 1, 2, \dots, n, \forall j = 1, 2, \dots, z \\ c_i \in \{0, 1\}, \quad \forall i = 1, 2, \dots, z \end{array} \right.$$

The first constraint guarantees that every rectangle is covered. The second constraint ensures that exactly s guards are used. The third constraint associates each selected vertex with only one color. The fourth and sixth constraints establish if a color is used or not. Finally, the fifth constraint ensures that no rectangle is covered by more than one guard of the same color.

5.2 Extended Visibility

This extension is highly similar to the original problem, yet a guard may cover adjacent rectangles up to a distance D .

Let m be the number of rectangles to cover, n the number of vertices, D the maximum distance and d_{ij} the minimum distance for vertex i to rectangle j . Let

$$p_{ij} = \begin{cases} 1, & \text{if } d_{ij} \leq D \\ 0, & \text{otherwise} \end{cases}$$

d_{ij} can easily be computed as follows:

For each vertex i , the search is initialized with all rectangles incident to i , which form the rectangles at distance 0. Subsequently, a BFS is performed over the rectangle adjacency graph, giving us the minimum distance from vertex i to every rectangle j .

The decision variables are defined as follows:

- v_i indicates if vertex i contains a guard;

The objective is to minimize the total number of guards:

$$\sum_{i=1}^n v_i$$

subject to the following constraints:

$$\begin{cases} \sum_{i=1}^n v_i \times p_{ij} \geq 1, & \forall j = 1, 2, \dots, m \\ v_i \in \{0, 1\}, & \forall i = 1, 2, \dots, n \end{cases}$$

Chapter 6

Conclusion